

Visual Causal-chain Bookmarking: Retrieval-Weighted Credit Assignment for Reinforcement Learning from Delayed Reward

A dissertation submitted to The University of Manchester for the degree of
Master of Science in Robotics
in the Faculty of Science and Engineering

Year of submission

2026

Student ID

TODO-STUDENT-ID

School of Engineering

Contents

Contents	I
Abstract	III
Declaration of originality	IV
Copyright statement	V
Acknowledgments	VI
1 Introduction	1
2 Preliminaries	2
2.1 Markov decision processes and policy-gradient reinforcement learning	2
2.2 RUDDER: LSTM-based return redistribution	3
2.3 LOOP: leave-one-out advantage estimation with PPO-clipped updates	4
3 Technical contribution	4
3.1 VCBM for tabular MDPs: causal discovery and Welford estimation	5
3.1.1 Motivation	5
3.1.2 Causal-role discovery	5
3.1.3 Bookmark discovery and value estimation	6
3.1.4 Action selection	6
3.2 VCBM for language-based MDPs: retrieval-weighted credit blending	6
3.2.1 Motivation	6
3.2.2 Bookmark detection and episodic memory	7
3.2.3 Retrieval and temperature-scaled weighting	7
3.2.4 Blending with the trajectory-level LOO advantage	7
3.3 Where the two instantiations agree and where they necessarily diverge	8
4 Experiments	9
4.1 Controlled evaluation: RUDDER vs. VCBM in a delayed-reward tabular MDP	9
4.1.1 Environment	9
4.1.2 Method and metrics	9
4.1.3 Discovered causal structure	10
4.1.4 Interpretation	10
4.2 Large-scale evaluation: LOOP vs. VCBM-blended LOOP on AppWorld	11
5 Conclusions	11

References 13

Appendices 14

Word count: 3718

Abstract

Reinforcement learning (RL) agents trained with a single trajectory-level return face a structural credit-assignment problem: when a reward arrives many steps after the decision that caused it, every intermediate action receives the same learning signal, regardless of whether it was causally responsible for the outcome. This dissertation studies *Visual Causal-chain Bookmarking* (VCBM), a family of methods that concentrate credit on the specific decision points that determine an episode’s return rather than spreading it uniformly across the trajectory, and develops two domain-specific instantiations of this idea.

The first instantiation targets a fully observed, low-dimensional Markov decision process (a delayed-reward watch-repair task), where VCBM statistically discovers which state components are causally controlled by the agent’s actions and estimates per-decision-point action values directly; because the environment’s causal structure is verifiable, this instantiation is evaluated against known ground truth as well as against RUDDER [1], a return-redistribution baseline built on an LSTM contribution network. VCBM converges to a 95.07% correct-decision rate in a mean of $1,532.3 \pm 1,428.0$ episodes across 20 random seeds (range 1,196–7,757), against RUDDER’s slower and more variable convergence (mean $3,327.1 \pm 2,010.9$ episodes, median 2,829, range 1,603–10,936, computed from the 20 per-seed training logs), and its discovered causal structure exactly matches the environment’s true generative structure in all 20 seeds.

The second instantiation targets a partially observed, language-based setting – an LLM agent trained with Leave-One-Out Proximal Policy Optimisation (LOOP) [2] inside the AppWorld benchmark [3] – where causal discovery is intractable and a structural heuristic (state-mutating tool calls) substitutes for it, with credit distributed via cosine-similarity retrieval over an episodic memory of bookmarked turns and blended with LOOP’s trajectory-level advantage. TODO: state the headline quantitative result (e.g. convergence speed and/or final policy quality relative to the LOOP baseline) once the AppWorld LOOP-vs-VCBM CSF3 run completes; until then, this instantiation is a designed and implemented, but not yet empirically validated, contribution.

Taken together, the watch-repair result supports the dissertation’s central claim – that explicitly identifying and weighting the causally relevant steps of a trajectory is a more sample-efficient credit-assignment strategy than either an undifferentiated trajectory-level baseline or a learned return-redistribution network – in the one setting where it has so far been tested; TODO: extend this claim to the AppWorld setting once that comparison is complete.

Declaration of originality

I hereby confirm that no portion of the work referred to in the thesis has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Copyright statement

- i The author of this thesis (including any appendices and/or schedules to this thesis) owns certain copyright or related rights in it (the “Copyright”) and s/he has given The University of Manchester certain rights to use such Copyright, including for administrative purposes.
- ii Copies of this thesis, either in full or in extracts and whether in hard or electronic copy, may be made *only* in accordance with the Copyright, Designs and Patents Act 1988 (as amended) and regulations issued under it or, where appropriate, in accordance with licensing agreements which the University has from time to time. This page must form part of any such copies made.
- iii The ownership of certain Copyright, patents, designs, trademarks and other intellectual property (the “Intellectual Property”) and any reproductions of copyright works in the thesis, for example graphs and tables (“Reproductions”), which may be described in this thesis, may not be owned by the author and may be owned by third parties. Such Intellectual Property and Reproductions cannot and must not be made available for use without the prior written permission of the owner(s) of the relevant Intellectual Property and/or Reproductions.
- iv Further information on the conditions under which disclosure, publication and commercialisation of this thesis, the Copyright and any Intellectual Property and/or Reproductions described in it may take place is available in the University IP Policy (see <http://documents.manchester.ac.uk/DocuInfo.aspx?DocID=24420>), in any relevant Thesis restriction declarations deposited in the University Library, The University Library’s regulations (see <http://www.library.manchester.ac.uk/about/regulations/>) and in The University’s policy on Presentation of Theses.

Acknowledgments

Acknowledgments go here.

1 Introduction

Reinforcement learning (RL) agents that learn from a trajectory-level reward face a structural credit-assignment problem whenever that reward is *delayed* and *sparse*: it arrives only at the end of an episode, after dozens or hundreds of intermediate actions, most of which did not causally influence the outcome. This difficulty is not a corner case – it recurs across the trial-and-error paradigm’s main applications, from robotic control to the post-training alignment of large language models (LLMs) via reinforcement learning from human feedback (RLHF) [4], [5]. A watch-repair technician’s one decision to repair or not repair a watch determines the profit realised weeks later, after many unrelated logistics events; an LLM agent solving a multi-step task in a simulated software environment such as AppWorld [3] executes a long sequence of tool calls, of which only a handful actually change task-relevant state. In both cases, a policy-gradient update that assigns the same scalar advantage to every action in the trajectory is forced to average this delayed signal across many causally irrelevant steps, which slows convergence and can bias the policy towards whichever irrelevant actions happen to correlate with high-return trajectories.

Two established families of methods respond to this problem without questioning the underlying credit-assignment strategy. Trajectory-level baselines, such as the leave-one-out (LOO) advantage estimator used in LOOP [2] for training LLM agents, reduce the variance of a Monte-Carlo return by subtracting a baseline computed from other rollouts of the same scenario, but every token or action within a rollout still receives an identical, undifferentiated advantage. Return-redistribution methods such as RUDDER [1] instead train an auxiliary network – an LSTM that predicts the eventual return from every prefix of the trajectory – and redistribute reward as the difference between consecutive predictions, which in principle produces a denser and better-shaped reward signal. However, RUDDER’s redistribution is learned end-to-end from a recurrent model whose predictions are only as reliable as its own training data, so early in training, before the LSTM has converged, the redistributed reward is itself noisy, and the resulting variance can slow down the tabular or parametric policy that consumes it.

An underexploited alternative is to make credit assignment *structural* rather than purely statistical: identify which specific timesteps in a trajectory are causally responsible for the eventual outcome, and concentrate the learning signal on those timesteps directly, rather than learning a smooth redistribution over the whole sequence. This is the guiding idea behind *Visual Causal-chain Bookmarking* (VCBM), the method studied in this dissertation. VCBM is deliberately instantiated two different ways depending on what is tractable in the target environment. In a fully observed, low-dimensional Markov decision process (MDP), a genuine statistical test can determine which state components are causally controlled by the agent’s action, so VCBM performs explicit *causal discovery*: a two-proportion hy-

pothesis test classifies each state dimension as controlled, static, clock-like, or environmental noise, decision points (“bookmarks”) are identified as states matching a previously observed controlled-component change, and a Welford online estimator accumulates the return statistics of each bookmarked decision directly. In a partially observed, natural-language setting such as an LLM agent operating inside AppWorld, this statistical discovery procedure is intractable – the state is unstructured text, not a fixed set of interpretable components – so the second instantiation substitutes a structural heuristic (a turn is bookmarked if it issues a state-mutating tool call) for discovery, and replaces exact-match value lookup with retrieval: a lightweight text embedding of each bookmarked turn is stored in an episodic memory, and credit for a completed trajectory is distributed across bookmarks in proportion to their cosine similarity to the terminal state, blended with the trajectory-level LOO advantage that would otherwise be applied uniformly.

This dissertation’s technical contribution is the design, implementation, and empirical evaluation of both VCBM instantiations against their respective natural baselines. Section 2 formalises the shared MDP/RL notation and the two baseline algorithms – RUDDER’s LSTM-based return redistribution and LOOP’s leave-one-out PPO objective – that VCBM is compared against. Section 3 presents the full mathematical formulation of both VCBM instantiations: the causal-discovery-and-Welford-estimation mechanism for the tabular watch-repair task, and the retrieval-and-blend mechanism for the AppWorld LLM-agent task, making explicit where the two instantiations share a common structure and where they necessarily diverge. Section 4 reports controlled multi-seed experiments in the watch-repair environment, where VCBM’s discovered causal structure is directly verifiable against the environment’s known ground truth, together with TODO: the corresponding AppWorld LOOP-vs-VCBM comparison once training completes. Section 5 concludes with a statement of technical achievement against the dissertation’s original objectives and discusses the limitations of the structural-heuristic instantiation as a direction for future work.

2 Preliminaries

2.1 Markov decision processes and policy-gradient reinforcement learning

An episodic MDP is a tuple $(\mathcal{S}, \mathcal{A}, P, r, \gamma, T)$ with state space $\mathcal{S}$, action space $\mathcal{A}$, transition kernel $P(s_{t+1} \mid s_t, a_t)$, reward function $r_t = r(s_t, a_t)$, discount factor $\gamma \in (0, 1]$, and horizon T . A stochastic policy $\pi_\theta(a_t \mid s_t)$, parameterised by θ , generates a trajectory $\tau = (s_0, a_0, r_0, \dots, s_T)$ with realised

(undiscounted, as used throughout this dissertation’s environments) return

$$G(\tau) = \sum_{t=0}^T r_t. \quad (1)$$

Policy-gradient methods maximise $J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[G(\tau)]$ by ascending an estimate of $\nabla_\theta J(\theta) = \mathbb{E}_\tau[\sum_t \nabla_\theta \log \pi_\theta(a_t | s_t) A_t]$, where A_t is an *advantage* estimate that reduces gradient variance relative to using the raw return $G(\tau)$ directly. The two baselines studied in this dissertation, RUDDER and LOOP, both address the variance/credit-assignment problem inherent in estimating A_t from a sparse, trajectory-level reward, but with different mechanisms; the central hypothesis of this dissertation, tested via VCBM, is that a third mechanism – explicit identification of the causally relevant timesteps – offers a more sample-efficient path to a low-variance, well-shaped A_t than either.

2.2 RUDDER: LSTM-based return redistribution

RUDDER (RUDDER: Return Decomposition for Delayed Rewards) [1] addresses delayed, sparse reward by training an auxiliary sequence model to predict the eventual episode return from every prefix of the trajectory, then using the *change* in that prediction from one timestep to the next as a redistributed, per-step reward. Let $\Delta s_t = s_t - s_{t-1}$ denote the state-difference at timestep t . Given the trajectory’s state-difference and action sequence up to step t , an LSTM with disabled forget and output gates produces a return prediction at every prefix,

$$\hat{G}(s_{0:t}, a_{0:t}) = \text{Linear}(\text{LSTM}(\Delta s_{0:t}, a_{0:t}))_t, \quad (2)$$

trained against the true (scaled) episode return $G_0 = G(\tau)/R_{\text{scale}}$ with a combination of a terminal-step loss and an auxiliary loss applied at every prefix,

$$\mathcal{L}_{\text{RUDDER}} = \underbrace{(\hat{G}(s_{0:T}) - G_0)^2}_{\mathcal{L}_{\text{main}}} + \lambda_{\text{aux}} \underbrace{\frac{1}{T} \sum_{t=0}^T (\hat{G}(s_{0:t}) - G_0)^2}_{\mathcal{L}_{\text{aux}}}. \quad (3)$$

The redistributed reward at each timestep is then the scaled first difference of consecutive predictions,

$$\tilde{r}_t = R_{\text{scale}} \cdot (\hat{G}(s_{0:t}) - \hat{G}(s_{0:t-1})), \quad \hat{G}(s_{0:-1}) \equiv 0, \quad (4)$$

which sums telescopically to the true episode return, $\sum_t \tilde{r}_t = R_{\text{scale}} \cdot \hat{G}(s_{0:T}) \approx G(\tau)$, while ideally concentrating non-zero reward at the timesteps the LSTM has learned are predictive of the outcome. Because $\tilde{r}_t$ is only as accurate as the LSTM’s own convergence, RUDDER’s redistribution is a *learned, statistical* solution to credit assignment: it does not identify causal structure directly, but approximates it

through gradient-based function approximation, and is used in this dissertation as the representative baseline against which VCBM’s structural approach is compared in a tabular setting (Section 4).

2.3 LOOP: leave-one-out advantage estimation with PPO-clipped updates

LOOP addresses the same delayed-reward problem in a different regime – training an LLM policy via multi-turn tool-use rollouts, where a value-function critic is memory-expensive to maintain alongside a large language model. Given K rollouts $\tau^{(1)}, \dots, \tau^{(K)}$ sampled from the *same* task scenario, each with realised return $G^{(k)} = G(\tau^{(k)})$, the leave-one-out baseline for rollout k is the mean return of the *other* $K - 1$ rollouts,

$$b^{(k)} = \frac{1}{K-1} \sum_{j \neq k} G^{(j)}, \quad A^{(k)} = G^{(k)} - b^{(k)}, \quad (5)$$

which is applied uniformly to every token generated within rollout k – i.e. $A_t = A^{(k)}$ for all t in that rollout, the trajectory-level credit-assignment limitation that VCBM’s blend mechanism (Section 3) directly targets.

This uniform advantage is then consumed by a PPO-clipped policy-gradient objective, which is needed because rollouts are generated by a separate, possibly numerically divergent inference engine. The policy-gradient update requires an importance weight correcting for the mismatch between the log-probability under which a token was sampled, $\pi_{\theta_{\text{old}}}$, and the log-probability under the current training weights, π_{θ} : $w_t = \exp(\log \pi_{\theta}(a_t) - \log \pi_{\theta_{\text{old}}}(a_t))$. The resulting per-token objective applies a PPO-style clip [2] to this importance weight to bound the size of any single update,

$$\mathcal{L}_{\text{PPO}}(t) = -\min(w_t A_t, \text{clip}(w_t, 1 - \epsilon, 1 + \epsilon) A_t), \quad (6)$$

with clip range ϵ (default 0.1 in the AppWorld configuration used in this dissertation). Equation (6) is the loss into which VCBM’s retrieval-weighted, per-token advantage (Section 3.2) is substituted in place of the uniform $A_t = A^{(k)}$, so that VCBM modifies *which* advantage each token receives, without requiring any change to the surrounding PPO-clipped policy-gradient machinery.

3 Technical contribution

Both instantiations of VCBM developed in this dissertation share a common three-stage template – *detect* the decision points that plausibly carry causal weight, *store* them in a structured memory, and *weight* the learning signal according to that memory – but the concrete mechanism at each stage necessarily differs between a fully observed tabular MDP, where causal discovery is statistically

tractable, and a partially observed, language-based MDP, where it is not. This section presents both instantiations in full, and states explicitly, rather than implicitly, where the two diverge.

3.1 VCBM for tabular MDPs: causal discovery and Welford estimation

3.1.1 Motivation

In a fully observed MDP with a small, fixed-dimensional state space, it is possible to test directly, from interaction data alone, which state components an agent’s action can causally influence and which change independently of it. If such a test is available, credit for the final return can be assigned exactly to the decision points where a causally controlled change originates, rather than diffused across an entire trajectory of otherwise uninformative filler steps – as occurs, for instance, in the watch-repair environment of Section 4, where a single decision at $t = 0$ determines the return realised 50 steps later.

3.1.2 Causal-role discovery

Let the state be decomposed into n scalar components $s = (s^{(1)}, \dots, s^{(n)})$. For each component i and each action value $a \in \{0, 1\}$, a running count $c_{i,a}$ tracks how often component i changed value on a step where action a was taken, out of n_a total steps with that action. After a warm-up period, each component is classified into one of four causal roles using its empirical change rate $\rho_{i,a} = c_{i,a}/n_a$:

$$\text{role}(i) = \begin{cases} \text{CLOCK} & \min(\rho_{i,0}, \rho_{i,1}) > \eta_{\text{clock}} \\ \text{STATIC} & c_{i,0} + c_{i,1} = 0 \\ \text{CONTROLLED} & z_i > z_{\text{thresh}} \\ \text{NOISE} & \text{otherwise,} \end{cases} \quad (7)$$

where z_i is the two-proportion test statistic comparing the change rate under each action,

$$z_i = \frac{|\rho_{i,0} - \rho_{i,1}|}{\sqrt{\hat{p}(1 - \hat{p})(1/n_0 + 1/n_1)}}, \quad \hat{p} = \frac{c_{i,0} + c_{i,1}}{n_0 + n_1}. \quad (8)$$

A component is CONTROLLED only if it changes at a rate that depends significantly on the action taken; CLOCK and STATIC components change (almost) deterministically regardless of action and can never carry action-dependent credit; NOISE components change at a rate that does not depend significantly on the action, given the available sample size, and are excluded from the value-estimation key

described below. This test is what allowed VCBM, without any hand-specified labels, to recover the causal structure of the watch-repair environment reported in Section 4.

3.1.3 Bookmark discovery and value estimation

A state is flagged as a *bookmark opportunity* – a decision point worth tracking – if its projection onto the CONTROLLED and CLOCK components matches a context previously observed immediately before a CONTROLLED component changed value. At each bookmark, a compact key is formed by projecting the state onto its STATIC, CONTROLLED, and CLOCK components only (marginalising out NOISE, which by construction cannot carry action-relevant signal), and a Welford online estimator accumulates, per (key, action) pair, the mean and variance of the *full* episode return $G(\tau)$ observed whenever that bookmark/action pair occurred:

$$\mu_n = \mu_{n-1} + \frac{G(\tau) - \mu_{n-1}}{n}, \quad M_{2,n} = M_{2,n-1} + (G(\tau) - \mu_{n-1})(G(\tau) - \mu_n), \quad (9)$$

with standard error $SE = \sqrt{M_{2,n}/(n(n-1))}$. Because the key marginalises out noise components, samples from many episodes that differ only in their noise trajectory are correctly pooled into the same value estimate – a form of sample sharing unavailable to a plain per-state value table.

3.1.4 Action selection

At a bookmark, actions are sampled uniformly until both actions have accumulated at least $n_{\min}$ visits; thereafter, a two-sample z -test on the two actions’ Welford means, $z = |\mu_0 - \mu_1|/\sqrt{SE_0^2 + SE_1^2}$, determines whether the value difference is statistically significant. If significant, the higher-mean action is chosen greedily (with a small residual exploration probability $\epsilon_{\min}$); otherwise, an ϵ -greedy rule with decaying ϵ continues exploring. At non-bookmark timesteps, an action is drawn uniformly at random, since Equation (7) guarantees such steps do not causally affect the outcome once the classifier has converged.

3.2 VCBM for language-based MDPs: retrieval-weighted credit blending

3.2.1 Motivation

The causal-role test of Section 3.1 requires a fixed, low-dimensional, fully observed state representation whose components can be tracked and change-tested individually. An LLM agent operating in AppWorld [3] instead observes unstructured natural-language tool outputs, with no fixed component

decomposition and no tractable way to run a per-component hypothesis test online during training. The second VCBM instantiation therefore replaces statistical causal *discovery* with a structural *heuristic* for what counts as a bookmark, and replaces exact-match value lookup with similarity-based *retrieval*, since no two AppWorld episodes share an identical state trajectory the way watch-repair episodes do.

3.2.2 Bookmark detection and episodic memory

A trajectory turn is bookmarked if it issues a state-mutating tool call (as opposed to a read-only or no-op action), detected directly via the environment server’s execution interface with no additional network calls. Each bookmarked turn’s tool-output text x is embedded with a deterministic, dependency-free hashing-trick bag-of-words encoding g_ϕ into a fixed-dimension vector,

$$g_\phi(x)_j = \frac{1}{\|v\|_2} \sum_{\text{token} \in x} \mathbb{1}[h(\text{token}) = j], \quad h(\text{token}) = \text{int}(\text{MD5}(\text{token})[:8]) \bmod d, \quad (10)$$

and stored in an episodic memory bank as a tuple $(\text{turn_idx}, g_\phi(x), G(\tau))$ – the bookmark’s position, its text embedding, and the return of the trajectory it belonged to.

3.2.3 Retrieval and temperature-scaled weighting

Given a completed trajectory’s terminal-observation embedding $z_H = g_\phi(x_H)$, credit is distributed across the top- k most similar bookmarks retrieved from memory by cosine similarity,

$$\text{sim}(z_H, k_i) = \frac{z_H \cdot k_i}{\|z_H\| \|k_i\|}, \quad w_i = \frac{\exp(\text{sim}_i/\tau)}{\sum_{j \in \text{top-}k} \exp(\text{sim}_j/\tau)}, \quad (11)$$

with temperature τ controlling how sharply weight concentrates on the single most similar retrieved bookmark. Tokens belonging to a retrieved bookmarked turn receive weight w_i ; all other tokens receive a fixed floor weight $\alpha \in (0, 1)$ so that non-bookmarked turns are down-weighted but never entirely excluded from the gradient; the resulting per-token weight vector is renormalised to sum to the number of output tokens, preserving the overall gradient scale of the underlying loss.

3.2.4 Blending with the trajectory-level LOO advantage

Rather than replacing LOOP’s leave-one-out advantage outright, the per-token VCBM weight $\tilde{w}_t$ (Equation (11), renormalised) is combined with the uniform LOO advantage $A^{(k)}$ (Equation (5)) via a mixing coefficient $\beta \in [0, 1]$:

$$A_{\text{VCBM},t} = A^{(k)} \cdot \tilde{w}_t, \quad A_{\text{blend},t} = (1 - \beta) A^{(k)} + \beta A_{\text{VCBM},t}, \quad (12)$$

which reduces exactly to plain LOOP at $\beta = 0$ and to a fully bookmark-concentrated advantage at $\beta = 1$; $A_{\text{blend},t}$ is substituted directly for the uniform A_t in Equation (6). Rollouts containing zero bookmarked turns receive the unmodified scalar LOO advantage, since Equation (12) is undefined without at least one retrieval candidate.

3.3 Where the two instantiations agree and where they necessarily diverge

Both instantiations follow the detect/store/weight template and are motivated by the same hypothesis – that concentrating credit on causally relevant decision points outperforms an undifferentiated trajectory-level signal – but they are not the same algorithm, and this dissertation does not claim they are. Table 1 makes the divergence explicit: the tabular instantiation *discovers* its bookmarks through a statistical hypothesis test with no learned embedding or similarity search anywhere in the pipeline, whereas the AppWorld instantiation *assumes* a structural rule for what constitutes a bookmark and relies on embedding-space retrieval, absent from the tabular version, to generalise across episodes that never share an identical state. The two instantiations are best understood as two different answers to the same design question – “how should credit be concentrated when discovery is/is not statistically tractable” – evaluated independently against the strongest available baseline in each regime, rather than as one method validated twice.

Table 1. Mechanistic comparison of the two VCBM instantiations studied in this dissertation.

Aspect	Tabular (Section 3.1)	AppWorld/LLM (Section 3.2)
Bookmark detection	Statistical causal-role test (Eq. (7))	Structural rule (state-mutating tool call)
Memory content	Welford (μ, σ^2) per (key, action)	Raw (index, embedding, return) tuples
Retrieval mechanism	Exact key match	Cosine-similarity top- k (Eq. (11))
Underlying policy	Tabular Q -style controller	LoRA-adapted LLM, PPO-clipped (Eq. (6))
Fallback when data is scarce	Balanced random sampling	Flat floor weight α

4 Experiments

4.1 Controlled evaluation: RUDDER vs. VCBM in a delayed-reward tabular MDP

4.1.1 Environment

The watch-repair environment models a single consequential decision under delayed, stochastic feedback. At $t = 0$ the agent observes a randomly drawn watch **BRAND** $b \sim \text{Uniform}\{0, 1, 2, 3\}$ and chooses whether to repair it ($a = 0$, incurring an immediate stochastic cost $r^{\text{repair}} \sim \mathcal{N}(\mu_{\text{repair}}[b], \sigma_{\text{repair}}^2[b])$) or not ($a = 1$, no cost). For the remaining 50 timesteps, two independent **NOISE** counters increment stochastically at fixed, action-independent rates, and a deterministic **CLOCK** component advances every step; the agent’s action has no further effect on the environment after $t = 0$. Only if the watch was repaired does the episode yield a terminal reward at $t = 50$, equal to the brand’s resale price minus accumulated transport-condition costs and a stochastic transport penalty. Because prices, repair costs, and transport-cost sensitivities differ by brand, the return-maximising policy is brand-dependent (repairing is optimal only for brands 1 and 2), giving a known ground-truth optimal action per state that is used purely for evaluation, never as a training signal.

4.1.2 Method and metrics

Both RUDDER (Section 2.2, redistributing reward into a tabular direct- Q -estimation update) and VCBM (Section 3.1) were trained for 20 independent random seeds each, with training terminated once the moving average (750-episode window) fraction of optimal first-decisions reached the 95% target. Table 2 reports episodes-to-convergence, our primary sample-efficiency metric, alongside the final converged decision quality.

Table 2. Watch-repair environment: episodes required to reach 95% correct first-decisions, 20 seeds each. RUDDER statistics computed from the 20 individual per-seed training logs (Section 4.1); VCBM statistics taken from the method’s own multi-seed summary log.

Method	Seeds converged	Episodes to converge (range) ↓	Episodes to converge (mean $\pm$ std) ↓	Final good-decision % ↑
RUDDER	20/20	1,603 – 10,936	3,327.1 $\pm$ 2,010.9 (median 2,829)	95.07% (by construction of stopping rule)
VCBM	20/20	1,196 – 7,757	1,532.3 $\pm$ 1,428.0 (median 1,204.5)	95.07 $\pm$ 0.00%

4.1.3 Discovered causal structure

Across all 20 VCBM seeds, the causal-role classifier (Equation (7)) recovered an identical label assignment – CONTROLLED, NOISE, NOISE, STATIC, CLOCK – for the environment’s five state components (repaired-flag, two transport-condition counters, brand, timestep), exactly matching the environment’s true generative structure described above: the repaired-flag is the only component whose change rate depends on the agent’s action, the two transport counters change at fixed action-independent rates, the brand is fixed for the episode, and the timestep advances deterministically. This is direct evidence that VCBM’s discovery mechanism (Section 3.1) recovers ground-truth causal structure from interaction data alone, without being told in advance which of the five components matters.

4.1.4 Interpretation

Both methods reach the 95% target in all 20 seeds, but VCBM’s episodes-to-converge distribution has a lower minimum (1,196 vs. 1,603) and a lower maximum (7,757 vs. 10,936) than RUDDER’s, consistent with the hypothesis that concentrating the learning signal on the single causally relevant decision point converges at least as fast as, and with less tail risk than, learning a smooth LSTM-based redistribution over the full 50-step trajectory. TODO: this interpretation is currently qualitative; add a paired significance test (e.g. Mann–Whitney U, appropriate given the right-skewed, non-normal distributions evident from the mean/median gap in Table 2) to substantiate the comparison once RUDDER’s full 20-seed distribution is available in the same structured form as VCBM’s summary log.

4.2 Large-scale evaluation: LOOP vs. VCBM-blended LOOP on AppWorld

TODO: this subsection reports the AppWorld comparison between the plain LOOP baseline (Section 2.3, Qwen-2.5-7B-Instruct policy, LoRA-adapted, FSDP2 training, $\beta = 0$ in Equation (12)) and VCBM-blended LOOP ($\beta = 1$) on identical GPU allocation, task split, and hyperparameters, once the corresponding CSF3 training runs complete. At minimum this subsection should report, per method: mean episode return over training iterations (learning curve), final held-out evaluation success rate (`r1.eval` split, evaluated every 5 iterations per the run configuration), and wall-clock/sample efficiency to reach a fixed return threshold, mirroring the metrics reported for the tabular experiment in Section 4.1 so the two experiments are directly comparable in structure even though their absolute metrics differ.

5 Conclusions

This dissertation set out to test whether reinforcement learning credit assignment can be improved by explicitly identifying the causally relevant decision points in a trajectory, rather than relying solely on trajectory-level baselines (as in LOOP) or learned return redistribution (as in RUDDER). Two domain-appropriate instantiations of this idea, jointly termed Visual Causal-chain Bookmarking (VCBM), were designed, implemented, and evaluated: a statistical causal-discovery-and-value-estimation mechanism for a fully observed tabular MDP, and a structural-heuristic-plus-retrieval mechanism for a partially observed LLM-agent setting.

In the tabular watch-repair environment, VCBM recovered the environment’s true causal structure from interaction data alone across all 20 evaluated seeds and converged to the target decision quality with a narrower and lower episode-count distribution than the RUDDER baseline (Section 4.1), directly supporting the dissertation’s central hypothesis in a setting where ground truth is fully verifiable. This evidence is currently confined to that one setting: TODO: once the AppWorld LOOP-vs-VCBM comparison (Section 4.2) completes, this paragraph must state explicitly whether the retrieval-weighted blend produced a measurable improvement over plain LOOP, since Technical Achievement is assessed primarily on demonstrated, not merely designed, contribution.

The dissertation is explicit, rather than silent, about a limitation of the AppWorld instantiation: unlike the tabular version, its bookmark detector is a hand-specified structural rule (state-mutating tool calls), not a learned or statistically discovered criterion, so it cannot be assumed to generalise to task domains where the causally relevant actions are not synonymous with state-mutating calls – a text-only reasoning task with no external tool-use, for instance, would have no bookmarks under the

current rule at all. A natural direction for future work is a learned bookmark detector for the language setting, trained to predict causal relevance from the same kind of interaction data the tabular classifier already exploits (Equation (7)), which would close the conceptual gap between the two instantiations identified explicitly in Table 1 and bring the language-agent instantiation closer to a genuine causal-discovery mechanism rather than a structural proxy for one.

References

- [1] J. A. Arjona-Medina et al., ‘RUDDER: Return decomposition for delayed rewards,’ *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, TODO: verify exact page range / proceedings volume number (cited on pp. III, 1, 3).
- [2] J. Schulman et al., *Proximal policy optimization algorithms*, 2017. arXiv: 1707.06347 (cited on pp. III, 1, 4).
- [3] H. Trivedi et al., ‘AppWorld: A controllable world of apps and people for benchmarking interactive coding agents,’ *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024, TODO: verify venue/page details – also confirm exact author list order from appworld.dev (cited on pp. III, 1, 6).
- [4] P. F. Christiano et al., ‘Deep reinforcement learning from human preferences,’ *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, TODO: verify exact page range (cited on p. 1).
- [5] Y. Bai et al., *Training a helpful and harmless assistant with reinforcement learning from human feedback*, Anthropic RLHF paper. TODO: confirm this (rather than the Constitutional AI paper, Bai et al. 2022b, arXiv:2212.08073) is the intended “Claude RLHF paper” citation – both are Anthropic RLHF-adjacent but describe different methods., 2022. arXiv: 2204.05862 (cited on p. 1).

Appendices

Only add the project outline and risk assessments as defined in the first deliverable for this unit.